



Universidad Nacional de Rosario

Facultad de Ciencias Exactas, Ingeniería y Agrimensura

Trabajo práctico de Sistemas Operativos I

Estudiantes:

Angelo Amadio

Dante Dahuach

Laurentino Luna

Federico Resels

Docentes:

Taihú Piré

Erica Vidal

Alejandro Rivosecchi

Tomás Castro

Introducción

El objetivo de este trabajo es simular un entorno de trabajo distribuido, gestionando recursos de una red de computadoras, con la intención de imitar un cluster.

Además, simula un entorno planificador-cliente, donde un proceso toma el rol de planificador de pedidos al cluster, y otros simulan ser clientes que constantemente necesitan recursos.

En este informe se busca exponer y explicar la implementación, el diseño, problemáticas encontradas en la realización de este trabajo y una solución al problema de compartir recursos en una red de manera concurrente.

Roles de cada participante

Cada uno de los participantes tomó los siguientes roles:

- Angelo Amadio: a cargo de la gestión de recursos y estado del agente C; encargado de manejar la asignación y liberación de recursos entre los agentes del cluster y los clientes Erlang.
- Dante Dahuach: a cargo de la documentación, integración, pruebas e interoperabilidad entre los procesos Erlang y C.
- Laurentino Luna: a cargo de comunicaciones en C; encargado de manejar el agente C usando epoll, lectura y escritura no bloqueantes y detección de cierres.
- Federico Resels: a cargo del planificador en Erlang; encargado de generar jobs, prevenir deadlocks y conexión con el agente C.

Diseño del sistema

Contamos con dos partes:

Agente C

El agente C simula un gestor de recursos, los cuales obtiene de nodos que se anuncian periódicamente. Contiene una tabla de nodos, una tabla de jobs activos, una cola de jobs denegados y una lista de jobs que se quedaron sin tiempo.

A su vez, cuenta con tres hilos que cumplen distintas funciones: un hilo que maneja las conexiones con epoll, un hilo que monitorea la cola para evitar deadlocks, y un hilo que trabaja sobre la cola atendiendo peticiones.

Se comunica de tres formas diferentes, en dos contextos distintos respectivamente:

Agente C ⇔ Scheduler Erlang

El programa de Erlang se comunica localmente a través de un puerto fijo con el agente C, que está alojado en la misma computadora.

El recorrido del pedido de un cliente tiene la siguiente forma:

Cliente ⇒ Scheduler Erlang ⇒ Agente C ⇒ Peticiones a los demás agentes

Agente C ⇔ Agente C

En este caso, se divide en dos secciones:

- Usando TCP: se realiza una comunicación entre dos agentes del cluster. El agente C que haya recibido un pedido de la forma **JOB_REQUEST** mandará **RESERVE** a los agentes correspondientes y esperará **GRANTED** o **DENIED**. Si recibe un pedido de la forma **JOB_RELEASE** mandará **RELEASE** a los agentes correspondientes.
- Usando UDP: se realiza una comunicación one-to-all, en la que un agente C se anuncia a sí mismo y sus recursos. Los agentes C receptores de este anuncio lo añaden a su tabla de nodos.

Erlang

El scheduler de Erlang está encargado de simular dos roles:

- El/Los cliente/s: Una simulación de usuario que pide una cantidad aleatoria de recursos (dentro de un rango) al scheduler para poder realizar un trabajo.
- El Scheduler: Un programa encargado de la gestión de pedidos realizados por el/los cliente/s y la comunicación con el agente de C.

Durante su ejecución, el scheduler le solicita al agente de C la información de cada uno de los nodos disponibles, espera pedidos de clientes, y calcula la validez de los pedidos de los clientes.

En caso de que llegue un pedido, el scheduler guarda la información del cliente, su pedido y crea un nuevo proceso que se encarga de gestionar el pedido junto con el agente C.

El cliente simula trabajos haciendo una espera inactiva durante unos segundos.

Agente C ⇒ Scheduler Erlang

El scheduler cuenta con un proceso “receptor”, que se encarga de recibir y procesar la información que envía el agente de C, y re-enviarla a los distintos procesos que la pidieron.

Scheduler Erlang ⇒ Agente C

Gracias al funcionamiento de la operación “gen_tcp:send”, es seguro permitir que cada proceso de Erlang envíe información concurrentemente.

Problemas encontrados y soluciones

Agente C

- Problema: manejar desconexiones inesperadas.
Solución: buscamos el nodo que se desconectó, y liberamos los jobs que dependían de este nodo. Luego, eliminamos el nodo de la tabla.
- Problema: reflejar el estado de los jobs en su tabla.
Solución: a la hora de manejar peticiones, si se trata de un nuevo job o de asignación de recursos, añadimos el job a la tabla. Respecto a la liberación de un job, dependiendo si tiene o no recursos, se actualiza o se elimina de la tabla.
- Problema: atender solicitudes encoladas en orden en **RELEASE**
Solución: cuando ocurre un **RELEASE** protegemos el acceso a la cola de jobs, y llamamos a **handle_job_request** con el primer trabajo encolado (el inicio de la cola) hasta que la cola esté vacía. Notemos que así evitamos la recursión.

Erlang

- Problema: gestionar múltiples pedidos de Jobs de parte de los clientes.
Solución: se utiliza una cola para guardar los pedidos a medida llegan, un mapa para guardar los PIDs de los clientes, y se crea un proceso “manager” para cada pedido de cliente.
- Problema: recibir y repartir mensajes de parte del Agente C.
Solución: se creó un nuevo proceso “message_manager” encargado de recibir toda la información enviada por el Agente C, procesarla y repartirla a cada proceso que la necesita, este mismo guarda un mapa de todos los managers de pedidos.
- Problema: procesar información de nodos enviada por el Agente C.
Solución: se utiliza una mezcla de funciones fold y map, para transformar una cadena de información de nodos en múltiples listas (de cadenas o enteros) o datos totales.

Prevención de deadlocks

Para justificar su funcionamiento, vamos a utilizar el ejemplo brindado en el enunciado del trabajo práctico, haciendo el seguimiento de cómo nuestra implementación lo evita.

Con dos nodos:

- Nodo A: 2 CPUs, 8 GB RAM, 0 GPU.
- Nodo B: 2 CPUs, 4 GB RAM, 1 GPU.

Jobs:

- Job1 (desde A): necesita 2 CPUs de A y 1 GPU de B.
- Job2 (desde B): necesita 1 GPU de B y 2 CPUs de A.

Si ambos planificadores envían solicitudes casi al mismo tiempo:

- Agente A concede 2 CPUs a Job1.
- Agente B concede 1 GPU a Job2.

En nuestra implementación, al realizar el chequeo de si recibimos:

- Desde A: GRANTED o DENIED de B. Dado que la cantidad de GPU de B ya fue concedida, el agente A recibe DENIED. Luego, no recibe todos los GRANTED que necesita, entonces encola Job1 y libera la CPU reservada.
- Desde B: GRANTED o DENIED de A. Dado que la cantidad de CPU de A ya fue concedida, el agente B recibe DENIED. Luego, no recibe todos los GRANTED que necesita, entonces encola Job2 y libera la GPU reservada.

Observación:

Nuestra implementación tiene preemption (ya que si no es posible otorgar todos los recursos que un job necesita, estos son devueltos a los nodos que los prestaron).

Luego, las colas de ambos agentes se ven como:

Cola de Agente A

[Job1] ⇒ []

Cola de Agente B

[Job2] ⇒ []

Consideremos todos los casos:

Caso 1: sleep_time de Agente A < sleep_time de Agente B

Se desencola Job1, y como los recursos están disponibles, se realiza el job.

Luego, se desencola Job2 y se realiza cuando los recursos tomados por Job1 estén disponibles.

Caso 2: sleep_time de Agente A > sleep_time de Agente B

Se desencola Job2, y como los recursos están disponibles, se realiza el job.

Luego, se desencola Job1 y se realiza cuando los recursos tomados por Job2 estén disponibles.

Caso 3: sleep_time de Agente A == sleep_time de Agente B

Misma situación que la planteada, luego en algún momento se llega al caso 1 o al caso 2

Caso 4: Por alguna situación externa/desconexión, ambos jobs se vuelven imposibles

Al pasar más de 15 segundos ambos en sus respectivas colas, Job1 y Job2 son eliminados y el proceso Erlang notificado.

Secuencia de comunicación

El cliente Erlang envía un pedido de job al scheduler

⇒ El scheduler envía un JOB_REQUEST a su agente C local

Sección agente C como servidor:

⇒ Agente C recibe JOB_REQUEST

Caso 1: El Job se puede realizar:

⇒ Se añade el Job a la tabla de jobs activos.

⇒ Le responde al proceso Erlang "JOB_GRANTED"

Caso 2: El Job no se puede realizar:

⇒ Se encola el job y se liberan los recursos tomados.

⇒ Le responde al proceso Erlang "JOB_DENIED"

⇒ Agente C recibe JOB_STATUS

Caso 1: El Job está en la tabla de jobs activos :

⇒ Le responde al proceso Erlang "JOB_GRANTED"

Caso 2: El Job sigue en la cola:

⇒ Le responde al proceso Erlang "JOB_DENIED"

Caso 3: El Job pasó mucho tiempo sin poder realizarse:

⇒ Se elimina al Job de la cola de jobs pendientes.

⇒ Le responde al proceso Erlang "JOB_TIMEOUT"

⇒ El agente C recibe JOB_RELEASE <job_id>

⇒ El agente C libera los recursos relacionados al job (locales y remotos) y elimina al job de su lista de Jobs

Sección agente C como cliente:

⇒ Agente C recibe "RESERVE" de otro agente C servidor

Caso 1: Tiene el recurso

⇒ Reserva el recurso localmente, le manda "GRANTED"
al agente C que actúa como servidor

Caso 2: No tiene el recurso

⇒ Envía al agente C que actúa como servidor "DENIED"